

Visual Tracking

The simulated setup will have:

- A camera mounted on two rotating joints.
- Pan: rotates the camera left and right.
- Tilt: rotates the camera up and down.
- A visible target.
- A controller that tries to keep the target at the image centre.

This is called visual servoing: camera measurements provide feedback that controls physical motion.

↳ control objective

if the image is (1920×1080)

$$(c_x, c_y) = (960, 540)$$

Suppose detected target center:

$$(u, v) = (1200, 400)$$

$$e_u = u - c_x = 1200 - 960 = 240 \text{ px}$$

$$e_v = v - c_y = 400 - 540 = -140 \text{ px}$$

- $e_u > 0$: target is right of centre.
- $e_u < 0$: target is left of centre.
- $e_v > 0$: target is below centre.
- $e_v < 0$: target is above centre.

Remember: image error is useful for pan/tilt tracking
to rotate the cam. toward target

horizontal pixel error $\rightarrow$ pan controller
vertical pixel error $\rightarrow$ tilt controller

$\rightarrow$ longer chain is needed for
"vision-guided arm"
not for
this simpler first form
"image based visual servoing"

1. If the target appears to the right of the image centre, which joint should respond: pan or tilt?
2. If the target appears above the centre, is e_v positive or negative using our definition?
3. Once the target is exactly centred, what are e_u and e_v ?
4. Why can pan/tilt tracking work without calculating the target's position in millimetres?

1. pan

2. negative

3. 0, 0

4. goal is only make the target appear at center
this can be achieved simple form of
error checking

camera image

$\downarrow$

detect target centre (u, v)

$\downarrow$

$$u_{\text{pan}} = K_p e_u + K_i \int e_u dt$$

calculate pixel errors (e_u , e_v)

↓

pan PID and tilt PID

↓

ctrl commands

↓

camera rotates

↓

new camera image

↺

$$+ K_d \frac{de_v}{dt}$$

$$u_{\text{pan}} = K_p e_v + K_i \int e_v dt + K_d \frac{de_v}{dt}$$

> p-only
ex:

$$e_u = 240 \text{ px}, \quad K_p = 0.002 \frac{\text{command}}{\text{pixel}}$$

$$u_{\text{pan}} = 0.002(240) = 0.48$$

1. If $e_u = -150$ px with the same $K_p = 0.002$, what is the pan command?
2. As the target approaches the centre, what happens to the magnitude of the P command?
3. If the target crosses from right of centre to left of centre, what happens to the sign of the P command?
4. If the target is approaching the centre very quickly, what should the D term try to do?

1. -0.3

2. error=0 $\Rightarrow$ P-cmd=0

3. goes from positive to negative

4. acting like visual braking

> Concrete Simulated Pan/Tilt Mechanism

world

└ fixed cam. base

└ pan joint/body

└ tilt joint/body

└ Camera

`qpos[0] = pan angle`

`qpos[1] = tilt angle`

```
qvel[0] = pan angular velocity
qvel[1] = tilt angular velocity

ctrl[0] = pan actuator command
ctrl[1] = tilt actuator command
```

1. When the pan joint rotates, does the attached tilt joint and camera rotate with it?
2. When only the tilt joint rotates, does it rotate its parent pan base?
3. If the target is right of centre and the camera correctly pans toward it, should the target's pixel coordinate u increase or decrease?
4. If we secretly read the target's exact MuJoCo world coordinates instead of detecting it in the rendered image, would this still be genuine visual servoing?

1. yes, coz tilt is attached to pan, if it moves so does tilt reacts
2. NO
3. decrease
4. that's ground-truth, not detection.

> vision part needs to extract
the target from rendered img

MuJoCo renders RGB image

↓

OpenCV selects pixels within target colour range

↓

Binary mask

→ white px: "prob. the target"
black px: "not target"

↓

Find target blob

↓

Calculate its centre (u, v)

↓

Calculate (eu, ev)

→ from white region
calculate its
centroid $\rightarrow (u, v)$
(avg. central px)

output: detected T/F
center: (u, v)

area: # target px

Example with a 640×480 image:

$$(c_x, c_y) = (320, 240)$$

Detected target:

$$(u, v) = (400, 180)$$

Therefore:

$$e_u = 80, \quad e_v = -60$$

So the target is right of and above the image centre.

1. Does the colour detector output millimetres or pixel coordinates?
2. What does the white region of the binary mask represent?
3. If the detected area is zero, should we send $(0, 0)$ to the controllers as if that were a real target?
4. For the example above, which two camera motions are needed to face the target?

1. px coords.

2. "probably the target"

3. it's top-left corner, so the cam. chase it wrapply; instead, report: detection = F

4. panning right ; tilting upward

The PID terms now have visual meanings:

- P: reacts to how far the target currently is from image centre.
- I: accumulates an offset that remains for a long time.
- D: reacts to how quickly the target is moving across the image and provides braking.
- Saturation: limits pan and tilt commands to what the actuators can provide.

$$e_u = 80 \text{ px}, \quad \forall p = 0.005$$

$$p_{\text{pan}} = 0.4$$

Experiment

positive pan: $u \uparrow$ positive tilt: $v \downarrow$
negative pan: $u \downarrow$ negative tilt: $v \uparrow$

From neutral:

- Positive pan: $u : 454 \rightarrow 547$, so positive pan moves the target rightward.
- Negative pan: $u : 454 \rightarrow 372$, so negative pan moves it leftward.
- Positive tilt: $v : 149 \rightarrow 62$, so positive tilt moves the target upward.
- Negative tilt: $v : 149 \rightarrow 229$, so negative tilt moves it downward.

→ current errors

$$e_u = +134$$

$$e_v = -91$$

→ negative pan to reduce u
→ negative tilt to increase v

$$ctrl_p = -\lambda_{p,p} \cdot e_u$$

$$ctrl_t = +\lambda_{p,t} \cdot e_v$$

1. For $e_u > 0$, should pan `ctrl` be positive or negative?
2. For $e_v < 0$, should tilt `ctrl` be positive or negative?
3. Which equation needs the explicit minus sign: pan or tilt?
4. Why did we measure these signs instead of assuming them from the words "positive pan" and "positive tilt"?

1. negative

2. negative

3. pan

4.

because words like "positive pan" don't guarantee an image direction. The result depends on:

- The joint axis direction.
- The right-hand rotation convention.
- The camera orientation.
- The image-coordinate convention.

⇒ ρ visual tracking

time= 0.0s	error=(134, -91) px	ctrl=(-0.402, -0.273)	detected=True
time= 0.5s	error=(-65, -36) px	ctrl=(0.195, -0.108)	detected=True
time= 1.0s	error=(-7, -5) px	ctrl=(0.021, -0.015)	detected=True
time= 1.5s	error=(42, 4) px	ctrl=(-0.126, 0.012)	detected=True
time= 2.0s	error=(-38, 6) px	ctrl=(0.114, 0.018)	detected=True
time= 2.5s	error=(20, 3) px	ctrl=(-0.060, 0.009)	detected=True
time= 3.0s	error=(1, 1) px	ctrl=(-0.003, 0.003)	detected=True
time= 3.5s	error=(-9, 0) px	ctrl=(0.027, 0.000)	detected=True
time= 4.0s	error=(11, 0) px	ctrl=(-0.033, 0.000)	detected=True
time= 4.5s	error=(7, 0) px	ctrl=(0.021, 0.000)	detected=True

```

time= 4.5s | error=( -7, 0) px | ctrl=( 0.021, 0.000) | detected=True
time= 5.0s | error=( 1, 0) px | ctrl=(-0.003, 0.000) | detected=True
time= 5.5s | error=( 2, 0) px | ctrl=(-0.006, 0.000) | detected=True
time= 6.0s | error=( -4, 0) px | ctrl=( 0.012, 0.000) | detected=True
time= 6.5s | error=( 2, 0) px | ctrl=(-0.006, 0.000) | detected=True
time= 7.0s | error=( -1, 0) px | ctrl=( 0.003, 0.000) | detected=True
time= 7.5s | error=( -1, 0) px | ctrl=( 0.003, 0.000) | detected=True

```

1. Which output proves that pan crossed beyond the image centre?
2. Why did the pan command reverse sign after 0.5 seconds?
3. Which axis settled faster: pan or tilt?
4. Why can P reach nearly zero error here, unlike the gravity-loaded pendulum?
5. Did the system use the target's ground-truth position anywhere in this loop?

1. second output. cuz it went from positive to negative
2. because model went to left
3. tilt
4. cuz, here there is no need for constant torque that balances gravity which causes steady-state error
5. no.

$\Rightarrow$ D controller

$$\hat{e} = \frac{e_{\text{current}} - e_{\text{prev.}}}{\Delta t}$$

$$= \text{ctrl}_{\text{pan}} = -(K_p e_u + K_d \hat{e}_u)$$

$$\text{ctrl}_{\text{tilt}} = K_p e_v + K_d \hat{e}_v$$

$\Rightarrow$ experiment

If $e_u > 0$ but rapidly decreasing, then:

- P is positive inside the parentheses.
- D is negative.
- D reduces the requested movement before the target crosses the centre.

Time	P horizontal error	PD horizontal error
0.0 s	+134 px	+134 px
0.5 s	-65 px	+2 px
1.0 s	-7 px	0 px
1.5 s	+42 px	0 px

1. Why was pan `ctrl` positive at 0.5 s even though $e_u = +2$?
2. Why was tilt `ctrl` nonzero at 1.0 s even though $e_v = 0$?
3. At 1.5 s, what do zero error, zero error rate and zero control collectively tell us?
4. Which controller gave the better transient response here: P or PD?

1. becu2 now p+d is considered as total; not just p
2. because rate has non zero px/s value so d creates it
3. target is centered, stationary in img, no act. cmd needed
4. PD

⇒ I term

Now, why might we need I if PD already works so well? In the current model, we don't—there is no constant external load. To learn I honestly, we need a persistent disturbance, such as:

- Cable tension pulling the camera.
- An unbalanced gimbal.
- Wind.
- Constant external torque.

Imagine a constant positive torque pushes pan away from centre. At steady state:

- Error rate becomes zero, so D becomes zero.
- If error were also zero, P would be zero.
- PD would then produce no torque to oppose the disturbance.

Therefore, PD must retain some pixel error so P can generate opposing torque. I can remember the persistent error and provide the required torque while bringing the current error back toward zero.

$$\text{data.gfrc} = 0.1$$

↳ apply const. external generalized force

$$e_u \hookrightarrow 0.1 \approx \frac{0.1}{0.003} \approx 33 \text{ px}$$

d: disturbance

$$\text{net } T = d - k_p e_u$$

$$0 = d - k_p e_u$$

$$e_u = \frac{d}{k_p}$$

$$\approx e_u = \frac{d}{\text{gear} \cdot k_p}$$

1. At steady state, if error rate is zero, which PD term can still oppose a constant disturbance?
2. If P must produce a nonzero opposing command, can the current error be exactly zero?
3. Which PID term can retain the needed command after the current error becomes nearly zero?

1. P term
2. no, steady-state error occurs so that P produce opposite cmd
3. I term

→ pd visual tracking

time= 0.0s	error=(134, -91) px	rate=(0.0, 0.0) px/s	ctrl=(-0.402, -0.273)	disturbance= 0.00	detected=True
time= 0.5s	error=(2, -5) px	rate=(-58.8, 58.8) px/s	ctrl=(0.023, 0.014)	disturbance= 0.00	detected=True
time= 1.0s	error=(0, 0) px	rate=(0.0, 29.4) px/s	ctrl=(-0.000, 0.015)	disturbance= 0.00	detected=True
time= 1.5s	error=(0, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.000, 0.000)	disturbance= 0.00	detected=True
time= 2.0s	error=(0, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.000, 0.000)	disturbance= 0.10	detected=True
time= 2.5s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True

time= 3.0s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True
time= 3.5s	error=(34, 0) px	rate=(29.4, 0.0) px/s	ctrl=(-0.117, 0.000)	disturbance= 0.10	detected=True
time= 4.0s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True
time= 4.5s	error=(34, 0) px	rate=(29.4, 0.0) px/s	ctrl=(-0.117, 0.000)	disturbance= 0.10	detected=True
time= 5.0s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True
time= 5.5s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True
time= 6.0s	error=(33, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.084, 0.000)	disturbance= 0.10	detected=True
time= 6.5s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True
time= 7.0s	error=(34, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.102, 0.000)	disturbance= 0.10	detected=True
time= 7.5s	error=(33, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.099, 0.000)	disturbance= 0.10	detected=True

$$e_u \Rightarrow \frac{0.1}{0.003} \approx 33 \text{ px}$$

1. Why does the controller settle near -0.1 control?
2. Why can D not eliminate the remaining 33-pixel error?
3. Approximately what is the net torque when control is -0.099 and disturbance is $+0.1$?
4. Could increasing K_p reduce the steady-state error? What possible trade-off have we already observed with larger P gains?

2. ctrl stays around -0.1 so it matches $+0.1$ disturbance

2. zero velocity $\Rightarrow D=0$

3- $0.099 \approx 0$

4. it may reduce ss error but it makes $\uparrow$ so it causes oscillation overshoot

$\rightarrow$ PID visual tracker

time= 0.0s	error=(134, -91) px	rate=(0.0, 0.0) px/s	ctrl=(-0.402, -0.273)	disturbance= 0.00	I=(0.000, 0.000)	detected=True
time= 0.5s	error=(2, -5) px	rate=(-58.8, 58.8) px/s	ctrl=(0.023, 0.014)	disturbance= 0.00	I=(0.000, 0.000)	detected=True
time= 1.0s	error=(0, 0) px	rate=(0.0, 29.4) px/s	ctrl=(-0.000, 0.015)	disturbance= 0.00	I=(0.000, 0.000)	detected=True
time= 1.5s	error=(0, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.000, 0.000)	disturbance= 0.00	I=(0.000, 0.000)	detected=True
time= 2.0s	error=(0, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.000, 0.000)	disturbance= 0.10	I=(0.000, 0.000)	detected=True
time= 2.5s	error=(31, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.104, 0.000)	disturbance= 0.10	I=(0.011, 0.000)	detected=True
time= 3.0s	error=(27, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.107, 0.000)	disturbance= 0.10	I=(0.026, 0.000)	detected=True
time= 3.5s	error=(22, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.089, 0.000)	disturbance= 0.10	I=(0.038, 0.000)	detected=True
time= 4.0s	error=(19, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.105, 0.000)	disturbance= 0.10	I=(0.048, 0.000)	detected=True
time= 4.5s	error=(15, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.087, 0.000)	disturbance= 0.10	I=(0.057, 0.000)	detected=True
time= 5.0s	error=(13, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.103, 0.000)	disturbance= 0.10	I=(0.064, 0.000)	detected=True
time= 5.5s	error=(11, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.102, 0.000)	disturbance= 0.10	I=(0.069, 0.000)	detected=True
time= 6.0s	error=(9, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.102, 0.000)	disturbance= 0.10	I=(0.075, 0.000)	detected=True
time= 6.5s	error=(8, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.103, 0.000)	disturbance= 0.10	I=(0.079, 0.000)	detected=True
time= 7.0s	error=(6, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.085, 0.000)	disturbance= 0.10	I=(0.082, 0.000)	detected=True
time= 7.5s	error=(5, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.085, 0.000)	disturbance= 0.10	I=(0.085, 0.000)	detected=True
time= 8.0s	error=(4, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.100, 0.000)	disturbance= 0.10	I=(0.088, 0.000)	detected=True
time= 8.5s	error=(4, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.102, 0.000)	disturbance= 0.10	I=(0.090, 0.000)	detected=True
time= 9.0s	error=(3, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.100, 0.000)	disturbance= 0.10	I=(0.091, 0.000)	detected=True
time= 9.5s	error=(3, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.102, 0.000)	disturbance= 0.10	I=(0.093, 0.000)	detected=True
time=10.0s	error=(2, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.085, 0.000)	disturbance= 0.10	I=(0.094, 0.000)	detected=True
time=10.5s	error=(2, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.101, 0.000)	disturbance= 0.10	I=(0.095, 0.000)	detected=True
time=11.0s	error=(2, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.102, 0.000)	disturbance= 0.10	I=(0.096, 0.000)	detected=True
time=11.5s	error=(1, 0) px	rate=(-29.4, 0.0) px/s	ctrl=(-0.085, 0.000)	disturbance= 0.10	I=(0.096, 0.000)	detected=True

this is reduced from const. ≈ 33 error cuz of I

So I does not magically erase error. It:

1. Accumulates the persistent positive error.
2. Builds the missing opposing command.
3. Allows the current P error to shrink.
4. Retains the required command even when current error is nearly zero.

1. At 2.5 seconds, which term supplies most of the opposing control: P or I?

2. Near 11 seconds, which term supplies most of it?
3. If we suddenly remove the disturbance while I still stores approximately 0.096, what might the camera initially do?
4. Why is the integral clamp useful?

2. P mostly ; Near 11.0s I mostly
2. I
3. overshoot or oscillate
4. prevents integral being too large (integral windup)

! - if target moves right $\rightarrow$ eu $\uparrow$ pan $\downarrow$

- cam joint angles can keep change slightly
even if the target is in the center

- moving target leaves the image == detected = false

$\rightarrow$ moving target tracker

time= 0.0s	error=(134, -91) px	rate=(0.0, 0.0) px/s	ctrl=(-0.402, -0.273)	joints=(0.0, 0.0)°	detected=True
time= 0.5s	error=(-1, -7) px	rate=(-29.4, 58.8) px/s	ctrl=(0.015, 0.008)	joints=(-10.6, -14.5)°	detected=True
time= 1.0s	error=(-2, -1) px	rate=(29.4, 0.0) px/s	ctrl=(-0.009, -0.003)	joints=(-5.4, -13.3)°	detected=True
time= 1.5s	error=(-1, 0) px	rate=(0.0, 0.0) px/s	ctrl=(0.003, 0.000)	joints=(-1.5, -19.4)°	detected=True
time= 2.0s	error=(0, 2) px	rate=(0.0, 29.4) px/s	ctrl=(-0.006, 0.021)	joints=(0.1, -18.4)°	detected=True
time= 2.5s	error=(2, 2) px	rate=(0.0, 0.0) px/s	ctrl=(-0.006, 0.006)	joints=(-0.6, -15.3)°	detected=True
time= 3.0s	error=(3, 2) px	rate=(0.0, -29.4) px/s	ctrl=(-0.009, -0.009)	joints=(-3.7, -11.0)°	detected=True
time= 3.5s	error=(3, 2) px	rate=(0.0, 0.0) px/s	ctrl=(-0.009, 0.006)	joints=(-8.3, -6.9)°	detected=True
time= 4.0s	error=(4, 1) px	rate=(0.0, 0.0) px/s	ctrl=(-0.012, 0.003)	joints=(-14.2, -3.8)°	detected=True
time= 4.5s	error=(3, 0) px	rate=(0.0, 0.0) px/s	ctrl=(-0.009, 0.000)	joints=(-20.2, -2.8)°	detected=True
time= 5.0s	error=(2, -1) px	rate=(0.0, 0.0) px/s	ctrl=(-0.006, -0.003)	joints=(-25.3, -4.2)°	detected=True
time= 5.5s	error=(1, -2) px	rate=(0.0, 0.0) px/s	ctrl=(-0.003, -0.006)	joints=(-28.7, -6.9)°	detected=True
time= 6.0s	error=(0, -2) px	rate=(0.0, 0.0) px/s	ctrl=(-0.006, -0.006)	joints=(-30.6, -10.6)°	detected=True
time= 6.5s	error=(-1, -2) px	rate=(0.0, 0.0) px/s	ctrl=(0.003, -0.006)	joints=(-30.7, -14.2)°	detected=True
time= 7.0s	error=(-2, -1) px	rate=(0.0, 0.0) px/s	ctrl=(0.006, -0.003)	joints=(-29.1, -16.5)°	detected=True
time= 7.5s	error=(-3, 0) px	rate=(0.0, 0.0) px/s	ctrl=(0.009, 0.000)	joints=(-25.5, -17.6)°	detected=True
time= 8.0s	error=(-4, 1) px	rate=(0.0, 0.0) px/s	ctrl=(0.012, 0.003)	joints=(-20.6, -16.6)°	detected=True
time= 8.5s	error=(-4, 2) px	rate=(-29.4, 0.0) px/s	ctrl=(0.027, 0.006)	joints=(-14.7, -13.8)°	detected=True
time= 9.0s	error=(-3, 2) px	rate=(0.0, 0.0) px/s	ctrl=(0.009, 0.006)	joints=(-9.0, -10.0)°	detected=True
time= 9.5s	error=(-2, 2) px	rate=(0.0, 0.0) px/s	ctrl=(0.006, 0.006)	joints=(-3.9, -6.0)°	detected=True
time= 10.0s	error=(0, 0) px	rate=(0.0, -29.4) px/s	ctrl=(-0.006, -0.015)	joints=(-0.6, -3.4)°	detected=True
time= 10.5s	error=(1, 0) px	rate=(0.0, 29.4) px/s	ctrl=(-0.003, 0.015)	joints=(0.2, -3.1)°	detected=True
time= 11.0s	error=(2, -2) px	rate=(0.0, 0.0) px/s	ctrl=(-0.006, -0.006)	joints=(-1.3, -5.2)°	detected=True
time= 11.5s	error=(3, -2) px	rate=(0.0, 29.4) px/s	ctrl=(-0.009, 0.009)	joints=(-5.1, -9.0)°	detected=True

$\pm 4px$ $\pm 2px$

> small image
error
does not mean
the robot is
stationary

1. Which column proves that the camera mechanism was actually moving?
2. Which column proves that it nevertheless kept the target centred?
3. At 6.5 seconds, pan was around -30.7° while horizontal error was only -1 pixel. What does that combination mean?
4. If we double the target's movement speed, what would you expect to happen to tracking error and required control effort?
5. Why is `detected=True` just as important as seeing a small numerical error?

1. joint col.
2. error col. proves

? It means the target had moved far enough that the camera needed about -30.7° pan to follow it, yet visual alignment

remained excellent at only -1 pixel error.

4. both

5. error is trustworthy only when "detected=true"

→ Target Loss Tracker

zero actuator
command not
freezing the
mechanism
instant

time= 0.0s	error=(134, -91)	rate=(0.0, 0.0)	px/s	ctrl=(-0.402, -0.273)	joints=(0.0, 0.0)°	detected=True
time= 0.5s	error=(-1, -7)	rate=(-29.4, 58.8)	px/s	ctrl=(0.018, 0.008)	joints=(-10.6, -1.5)°	detected=True
time= 1.0s	error=(-2, -1)	rate=(29.4, 0.0)	px/s	ctrl=(-0.009, -0.003)	joints=(-5.4, -8.3)°	detected=True
time= 1.5s	error=(-1, 0)	rate=(0.0, 0.0)	px/s	ctrl=(0.003, 0.000)	joints=(-1.5, 19.4)°	detected=True
time= 2.0s	error=(0, 2)	rate=(0.0, 29.4)	px/s	ctrl=(-0.000, 0.021)	joints=(0.1, 18.4)°	detected=True
time= 2.5s	error=(2, 2)	rate=(0.0, 0.0)	px/s	ctrl=(-0.006, 0.006)	joints=(-0.6, 15.3)°	detected=True
time= 3.0s	error=(3, 2)	rate=(0.0, -29.4)	px/s	ctrl=(-0.009, -0.009)	joints=(-3.7, 11.0)°	detected=True
time= 3.5s	error=(3, 2)	rate=(0.0, 0.0)	px/s	ctrl=(-0.009, 0.006)	joints=(-8.3, -6.9)°	detected=True
time= 4.0s	error=(4, 1)	rate=(0.0, 0.0)	px/s	ctrl=(-0.012, 0.003)	joints=(-14.2, -3.8)°	detected=True
time= 4.5s	error=(3, 0)	rate=(0.0, 0.0)	px/s	ctrl=(-0.009, 0.000)	joints=(-20.2, -2.8)°	detected=True
time= 5.0s	error= unavailable	rate=(0.0, 0.0)	px/s	ctrl=(0.000, 0.000)	joints=(-25.3, -4.2)°	detected=False
time= 5.5s	error= unavailable	rate=(0.0, 0.0)	px/s	ctrl=(0.000, 0.000)	joints=(-27.7, -4.8)°	detected=False
time= 6.0s	error= unavailable	rate=(0.0, 0.0)	px/s	ctrl=(0.000, 0.000)	joints=(-28.5, -4.9)°	detected=False
time= 6.5s	error= unavailable	rate=(0.0, 0.0)	px/s	ctrl=(0.000, 0.000)	joints=(-28.8, -4.9)°	detected=False
time= 7.0s	error=(0, -93)	rate=(0.0, 0.0)	px/s	ctrl=(-0.000, -0.279)	joints=(-28.0, -4.9)°	detected=True
time= 7.5s	error=(-3, -6)	rate=(0.0, 29.4)	px/s	ctrl=(0.009, -0.003)	joints=(-25.7, -16.9)°	detected=True
time= 8.0s	error=(-3, 1)	rate=(0.0, 0.0)	px/s	ctrl=(0.009, 0.003)	joints=(-20.6, -16.6)°	detected=True
time= 8.5s	error=(-3, 2)	rate=(29.4, 0.0)	px/s	ctrl=(-0.006, 0.006)	joints=(-14.6, -13.7)°	detected=True
time= 9.0s	error=(-3, 2)	rate=(0.0, 0.0)	px/s	ctrl=(0.009, 0.006)	joints=(-9.0, -10.0)°	detected=True
time= 9.5s	error=(-2, 2)	rate=(0.0, 0.0)	px/s	ctrl=(0.006, 0.006)	joints=(-3.9, -6.0)°	detected=True
time= 10.0s	error=(-1, 1)	rate=(0.0, 0.0)	px/s	ctrl=(0.003, 0.003)	joints=(-0.7, -3.4)°	detected=True
time= 10.5s	error=(1, 0)	rate=(0.0, 29.4)	px/s	ctrl=(-0.003, 0.015)	joints=(0.2, -3.2)°	detected=True
time= 11.0s	error=(3, -2)	rate=(29.4, 0.0)	px/s	ctrl=(-0.024, -0.006)	joints=(-1.3, -5.2)°	detected=True
time= 11.5s	error=(3, -3)	rate=(0.0, -29.4)	px/s	ctrl=(-0.009, -0.024)	joints=(-5.1, -9.0)°	detected=True

1. Why did `qpos` continue changing briefly while `ctrl=(0,0)` ?
2. Why did we reset the derivative history when detection was lost?
3. At 7 seconds, why was pan control zero but tilt control strongly negative?
4. If the target returned outside the camera's field of view, could this policy find it?
5. Does `ctrl=(0,0)` mean "actively hold the current pose," or merely "apply no motor command"?

1. becaz it had momentum, gradually went stop
2. if target undetectable we don't know its px/s
3. becaz $e_u = 0$; $e_\theta = -93$ camera needed to tilt upward
4. target lost → `ctrl = 0` → wait

If the target reappears outside the field of view, detection remains false. Because the camera only waits and performs no search movement, it cannot find the target. A different policy could slowly scan pan/tilt until detection returns.

5. apply no motor command

⇒ Conclusion

1. In one line, what does visual servoing mean?
2. Why does our pan equation have a minus sign while tilt has a plus sign?
3. What experimental evidence showed D improved tracking?

4. What experimental evidence showed I removed steady-state error?

5. What is the main limitation of our current target-loss policy?

1. controlling movement using visual feedback from cam.

2.

- Positive pan increased u , so pan must oppose e_u :

$$ctrl_{\text{pan}} = -PID(e_u)$$

- Positive tilt decreased v , so using the same sign as e_v corrects it:

$$ctrl_{\text{tilt}} = +PID(e_v)$$

3. pd visual tracker results

4. adding disturbance, pid visual tracker results

5. camera only waits and performs no search movement